

COMP101

Introduction to Programming

2025-26

Lecture-13

Selection Boolean – Python

Boolean tests – possible errors

#run-01 works when i/p=2

```
num = int(input("Enter an integer: ")) 2
if num ==2 or 3:
    print ("Number is 2 or 3")
else:
    print ("Not 2 or 3")
Number is 2 or 3
```

#run-02 works when i/p = 3

```
num = int(input("Enter an integer: ")) 3
if num ==2 or 3:
    print ("Number is 2 or 3")
else:
    print ("Not 2 or 3")
Number is 2 or 3
```

#run-03 – has a problem with i/p

```
num = int(input("Enter an integer: ")) 5
if num ==2 or 3:
    print ("Number is 2 or 3")
else:
    print ("Not 2 or 3")
Number is 2 or 3
```

Boolean tests – possible errors

#run-03 – has a problem

```
num = int(input("Enter an integer: ")) 5
if num ==2 or 3:
    print ("Number is 2 or 3")
else:
    print ("Not 2 or 3")
Number is 2 or 3
```

run-03 analysis:

'if' executes regardless of the input value.
Numbers outside range 2,3 should execute
'else' and output 'Not 2 or 3'.

The test is wrong:

The correct expression should be:

```
if num ==2 or num == 3:
```

Test #	i/p	o/p expected	o/p actual	comment
1	2	2 or 3	2 or 3	ok
2	3	2 or 3	2 or 3	ok
3	1	Not 2 or 3	2 or 3	err!
4	4	Not 2 or 3	2 or 3	err!
5	-2	Not 2 or 3	2 or 3	err!
6	-3	Not 2 or 3	2 or 3	err!

It is the logic or the syntax that is wrong
and is creating an inaccuracy

Implicit Casting

- Python uses 'implicit casting' for handling Boolean tests
 - uses the variable name as a condition
- Implicit casting can improve the logic and readability of code
 - But can give unexpected results
 - Useful technique but needs to be understood to be used properly
- Implicit casting is recommended in PEP8

Boolean implicit casting

- Variables are assigned data using the input() function
 - Data has a 'data type': int, float, string, Boolean ('bool')
 - Boolean evaluates True or False
- When a variable is assigned data, the variable evaluates True
 - User inputs a data value which is now stored in the var
 - var holds data so test = True (do 'if' and ignore 'else')
- When a variable has no data assigned, the variable evaluates False
 - User pressed return-key so no data is in the var
 - The var holds no data so test = False (ignore 'if' and do 'else')
- Implicit casting uses the variable name as a condition
 - Can make a program more readable and improves the logic

Boolean implicit casting

Pseudocode:

1. input
 - 1.1 input name
2. if (data has been input) *#var is assigned some data*
 - 2.1 print name
3. else *#return-key pressed – var not assigned*
 - 3.1 print null message

```
a_name = input ("Enter your name: ")
```

```
if a_name:  
    print ("Your name is ", a_name)  
else:  
    print ("Nothing input")
```

A var is initially empty.

When the var has data assigned to it, it is not empty.
It doesn't matter what the data is – it just has to have something in it.

When data is assigned to the var:

'if' evaluates True

we execute the if and ignore the else

When data is not assigned to the var (var is empty)

'if' evaluates False

we ignore 'if' and execute 'else'

`a_name` is a variable;

Python implicitly casts a Boolean type to it.

The expression checks if there is data in the var:

- var with data evaluates T
- var without data evaluates F

Boolean implicit casting

- Test table

Test #	i/p (all string)	o/p expected	o/p actual	comment
1	test1	if executes	if	var has data
2	test2	if executes	if	var has data
3	3	if executes	if	var has data
4	<return>	else executes	else	var is empty
...	...	...	...	...

Identify a var's data type with: `print (type(var_name))` - tells us what data type it is at that moment

Boolean implicit casting with 'bool'

#implicit Boolean casting – done by Python automatically

#recommended way to do it – PEP standard

```
a_name = input("Enter your name: ")
```

```
if a_name:
```

```
    print(Name is “,a_name)
```

```
else:
```

```
    print(“Nothing input”)
```

#explicit Boolean casting - #we 'tell' Python the var is a Boolean

```
if bool(a_name) == True:
```

```
    print("Name is “, a_name )
```

```
else:
```

```
    print("Nothing entered")
```

#equivalent code

```
if bool(a_name): #explicit casting
```

is the same as

```
if (a_name):#implicit casting (better than 'bool')
```

#errors

```
if (a_name) == True: #'bool' is missing
```

always evaluates False

The var is not cast as a bool so 'if' never executes

```
if (a_name) == 'True':
```

'True' is a string

'if' only executes when user inputs the string 'True'

'if bool' can be useful in complicated data, e.g.
testing for T when a value is in a data set etc.

Boolean Operators: Implicit casting

#assign variable 'flag' as Boolean variable to take values True or False

```
flag = False
```

```
if not flag:
```

```
    print ("flag is True")
```

```
else:
```

```
    print ("flag is False")
```

#written in the positive – easier to understand

```
flag = True
```

```
if flag:
```

```
    print ("flag is True")
```

```
else:
```

```
    print ("flag is False")
```

'if not flag' means 'if not False' means 'it is not False so it must be True'

Debugging hint:

Casting vars to a different type can get confusing as what the type has been cast to (int, float, string, bool).

Identify a var's type with: `print (type(var_name))` - tells us what data type it is at this moment

Boolean variables – use ‘natural’ var names

#input a year: if divisible exactly by 4 it is a leap year

```
year = int(input("Enter a year: "))
```

isALeapYear evaluates T or F depending on (year MOD 4) leaving no remainder

```
isALeapYear = (year%4 == 0)    #% is MOD division
```

```
if isALeapYear:                #test reads more naturally
```

```
    print ("Year is divisible by 4 – it is a leap year")
```

```
else:
```

```
    print ("No leap year")
```

Boolean variables – use ‘natural’ var names

```
year = int(input('Input year: '))
if (year%4==0):      #no use of Boolean
    print ('leap year')
else:
    print ('No leap year')
```

```
year = int(input('Input year: '))
isALeapYear = (year%4==0)      #calc for a leap year and assign to a new var 'isALeapYear' for readability
if isALeapYear:                #new var used as Boolean var in if statement
    print ('leap year')
else:
    print ('No leap year')
```

'in' operator

We can test if a number is in a set of numbers

#W4L13S11

```
num = int(input("Enter a number: "))
if num in (2,3):
    print ("Number is 2 or 3")
else:
    print ("Number not in the set")
```

#previous example without 'in' operator

```
num = int(input("Enter a number: "))
if num ==2 or num == 3:
    print ("Number is 2 or 3")
else:
    print ("Not 2 or 3")
```

'in' operator

- 'in' operator is useful when we have a larger set of numbers
- we want to check a specific number has been entered
- without 'in' long sets of numbers make coding harder and confusing

#w4l13s13

```
odd = int(input("Enter an odd number between 1 and 10: "))
```

```
if odd in (1,3,5,7,9):
```

```
    print ("Number is odd")
```

```
else:
```

```
    print ("Number not odd")
```

#previous example without 'in' operator

```
num = int(input("Enter a number: "))
```

```
if num==1 or num==3 or num==5 or num==7 or num==9:
```

```
    print ("Number is odd")
```

```
else:
```

```
    print ("Number not odd")
```

'in operator'

- Test table – input of odd numbers

Test #	i/p	o/p expected	o/p actual	comment
1	1	odd	odd	ok
2	9	odd	odd	ok
3	5	odd	odd	ok
4	2	not odd	not odd	ok
5	10	not odd	not odd	ok
6	11	not odd	not odd	ok – not in the set

'in range'

- Useful to see if a number is in a large set of numbers

#w4l13s15

```
num = int(input("Enter an integer: "))
```

```
if num in range (1,100):
```

```
    print ("Number is in range")
```

```
else:
```

```
    print ("Number not in range")
```

'in range' allows us to set up a range of sequential numbers and look for any number that is in the range.

Here, the start-value is 1 and the end-value is 100

Hence we can test if user has input any integer between 1 and 100

However . . . See test table next slide

'in range' operator

- Test table – range

Test #	i/p	o/p expected	o/p actual	comment
1	1	in range	in range	ok
2	50	in range	in range	ok
3	99	in range	in range	ok
4	100	in range	not in range	range is 0-99
5	101	not in range	not in range	ok
6	a	err message	err message	ok

Test#4 is not in error:

Computing starts a numeric sequence at index zero [0]:
Hence: integer range (1, 100) is held in index positions [0], [99]

i.e. the last integer in the range (1, 100) is not part of the integer range 1 to 100:
Actual integers are 0 to 99